

Régression logistique en R

Table of contents

0.1 Régression logistique	1
-------------------------------------	---

0.1 Régression logistique

Pour tirer une classification d'une régression logistique en R, il faut faire une étape supplémentaire après la régression proprement dite : il faut convertir les probabilités en prédictions à l'aide du choix d'un *seuil*.

Commençons par charger les données :

```
# Chargement des données et traitement de la colonne d'intérêt

# Choisir le fichier des cartes de crédit
mydata <- read.table(file.choose(), header=TRUE, sep=";",
                     na.strings="NA", dec=".", strip.white=TRUE)

mydata$Default.Next.Month = factor(mydata$Default.Next.Month)
```

On fait la régression à l'aide de `glm` :

```
model <- glm(Default.Next.Month ~ EDUCATION + SEX + MARRIAGE + AGE,
             data = mydata, family = binomial)
summary(model)
```

Call:

```
glm(formula = Default.Next.Month ~ EDUCATION + SEX + MARRIAGE +
    AGE, family = binomial, data = mydata)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-0.872667	0.108472	-8.045	8.62e-16	***
EDUCATION	0.077997	0.017675	4.413	1.02e-05	***
SEX	-0.202746	0.028424	-7.133	9.82e-13	***
MARRIAGE	-0.109947	0.029530	-3.723	0.000197	***
AGE	-0.001111	0.001681	-0.661	0.508889	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 31705 on 29999 degrees of freedom
Residual deviance: 31619 on 29995 degrees of freedom
AIC: 31629

Number of Fisher Scoring iterations: 4

Ensuite on fixe un seuil (*threshold* ou *cutoff* dans la littérature) et on calcule les prédictions de classes à l'aide de ce seuil :

```
threshold = 0.25
predictions <- predict(model, type = "response")
predicted_classes <- factor(ifelse(predictions > threshold, 1, 0), levels=c(0,1))
```

On compare avec les valeurs présentes dans les données :

```
library(caret)
```

Le chargement a nécessité le package : ggplot2

Le chargement a nécessité le package : lattice

```
confusionMatrix(predicted_classes, mydata$Default.Next.Month)
```

Confusion Matrix and Statistics

	Reference	
Prediction	0	1

```
0 20278 5547
1 3086 1089
```

```
Accuracy : 0.7122
95% CI : (0.7071, 0.7174)
No Information Rate : 0.7788
P-Value [Acc > NIR] : 1
```

```
Kappa : 0.0369
```

```
McNemar's Test P-Value : <2e-16
```

```
Sensitivity : 0.8679
Specificity : 0.1641
Pos Pred Value : 0.7852
Neg Pred Value : 0.2608
Prevalence : 0.7788
Detection Rate : 0.6759
Detection Prevalence : 0.8608
Balanced Accuracy : 0.5160
```

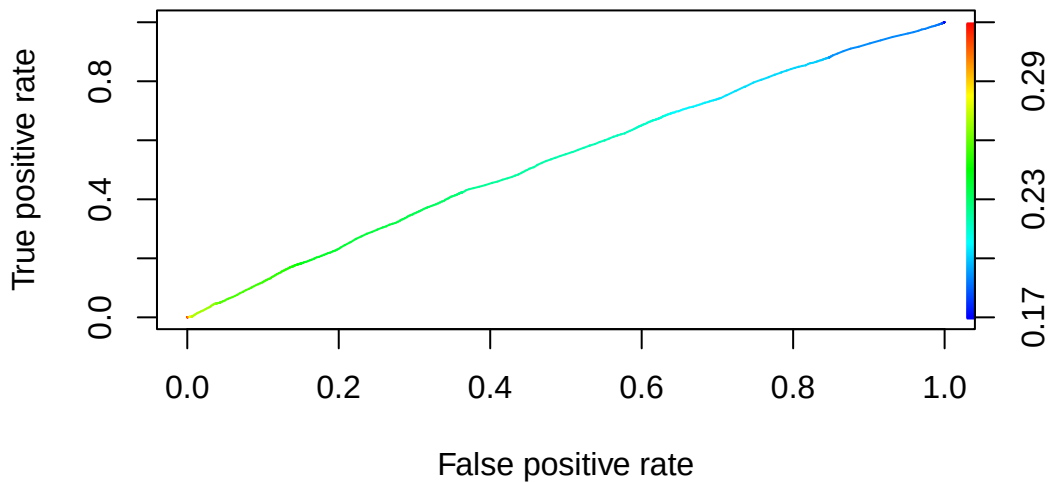
```
'Positive' Class : 0
```

Note : Ici j'ai simplifié le processus, d'habitude on sépare les données en un jeu d'entraînement et un jeu de test.

```
library(ROCR)
```

Warning: le package 'ROCR' a été compilé avec la version R 4.4.2

```
pred <- prediction(predictions, mydata$Default.Next.Month)
perf <- performance(pred, "tpr", "fpr")
plot(perf, colorize = TRUE)
```



Idéalement, la courbe ROC devrait partir de (0,0), monter presque verticalement jusqu'à la hauteur 1, puis continuer presque horizontalement jusqu'à (1,1). Le seuil optimal se trouve quelque part près du point (0,1) : on veut très peu de faux positifs (x proche de 0) et beaucoup de vrais positifs (y proche de 1).

Ce n'est pas du tout le cas ici, la courbe est presque plate, donc on sait que le taux d'erreur sera important quel que soit le choix de seuil.

On peut choisir sur la courbe ROC *le point le plus proche de (0,1)* de la façon suivante :

```
opt.cut <- function(perf, pred){
  cut.ind <- mapply(FUN=function(x, y, p){
    d <- (x - 0)^2 + (y-1)^2
    ind <- which(d == min(d))
    c(sensitivity = y[[ind]], specificity = 1-x[[ind]], cutoff = p[[ind]])
  }, perf@x.values, perf@y.values, pred@cutoffs)
}
optimal <- opt.cut(perf, pred)
print(optimal)
```

```
      [,1]
sensitivity 0.5295359
specificity 0.5262797
cutoff      0.2202317
```

On teste cette valeur :

```
threshold = optimal["cutoff",1]
predictions <- predict(model, type = "response")
predicted_classes <- factor(ifelse(predictions > threshold, 1, 0), levels=c(0,1))
confusionMatrix(predicted_classes, mydata$Default.Next.Month)
```

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	12423	3161
1	10941	3475

Accuracy : 0.5299
95% CI : (0.5243, 0.5356)
No Information Rate : 0.7788
P-Value [Acc > NIR] : 1

Kappa : 0.039

McNemar's Test P-Value : <2e-16

Sensitivity : 0.5317
Specificity : 0.5237
Pos Pred Value : 0.7972
Neg Pred Value : 0.2411
Prevalence : 0.7788
Detection Rate : 0.4141
Detection Prevalence : 0.5195
Balanced Accuracy : 0.5277

'Positive' Class : 0

Comme on le voit, ce point ne minimise pas forcément le taux d'erreur global. Il fait un compromis entre taux de faux positifs bas et taux de vrais positifs haut.

La **sensibilité** est la capacité à identifier correctement les vrais positifs. Elle se calcule comme le quotient (vrais positifs) / (vrais positifs + faux négatifs).

La **spécificité** est la capacité à identifier correctement les vrais négatifs. Elle se calcule comme le quotient (vrais négatifs) / (vrais négatifs + faux positifs).

Le **taux de faux positifs** est $1 - \text{spécificité}$.